

UTS
PENGOLAHAN CITRA



NAMA : Abdul Ghofur

NIM : 202331047

KELAS : D

DOSEN : Ir. Darma Rusjdi, M.Kom

NO.PC : 16

ASISTEN : 1. Fakhrol Fauzi Nugraha Tarigan
2. Muhammad Hanief Febriansyah
3. Clarenca Sweetdiva Pereira
4. Sakura Amastasya Salsabila Setiyanto

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2024/2025

DAFTAR ISI

DAFTAR ISI	2
BAB I	3
PENDAHULUAN	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II	4
LANDASAN TEORI	4
2.2 Model Warna HSV	4
2.3 Histogram Citra dan Analisisnya	4
2.4 Thresholding dan Segmentasi Warna	5
2.5 Peningkatan Kontras dan CLAHE	5
2.6 Perbaikan Citra Backlight	5
BAB III	6
HASIL	6
3.1 Soal Pertama	6
3.2 SOAL KEDUA	12
3.3 SOAL KETIGA	18
BAB IV	26
PENUTUP	26
DAFTAR PUSTAKA	27

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

- ☐ Bagaimana mendeteksi dan mengekstrak area warna merah, hijau, dan biru pada citra tulisan tangan?
- ☐ Bagaimana menentukan ambang batas (threshold) optimal untuk setiap warna?
- ☐ Bagaimana memperbaiki efek backlight sehingga area profil wajah menjadi lebih jelas dibanding latar belakang?

1.2 Tujuan Masalah

- ☐ Menerapkan segmentasi warna menggunakan model HSV untuk memisahkan komponen merah, hijau, dan biru.
- ☐ Menentukan dan mengurutkan nilai ambang batas warna terkecil hingga terbesar untuk masing-masing channel.
- ☐ Mengembangkan algoritma peningkatan kecerahan dan kontras khususnya pada area backlight agar profil lebih menonjol.

1.3 Manfaat Masalah

- ☐ Memahami konsep dan implementasi segmentasi warna dalam HSV untuk aplikasi komputer vision.
- ☐ Menguasai teknik thresholding otomatis yang dapat diterapkan pada berbagai tugas pengolahan citra.
- ☐ Mengimplementasikan metode peningkatan kontras (CLAHE) dan perbaikan backlight untuk meningkatkan kualitas visual citra.

BAB II

LANDASAN TEORI

2.1 Dasar-Dasar Pengolahan Citra Digital

Pengolahan citra digital merupakan proses manipulasi gambar digital melalui komputer untuk memperbaiki kualitas atau mengekstrak informasi tertentu. Gambar digital disimpan dalam bentuk matriks dua atau tiga dimensi, tergantung jumlah channel warnanya (Abdullah et al., 2021). Pada representasi 8-bit per channel, setiap piksel memiliki nilai 0–255.

Format paling umum adalah RGB (Red, Green, Blue) yang digunakan oleh sebagian besar sistem tampilan digital. RGB merupakan model warna aditif, di mana pencampuran tiga warna utama menghasilkan warna lain. Setiap piksel memiliki tiga komponen warna (merah, hijau, biru). Sementara itu, CMYK (Cyan, Magenta, Yellow, Key/Black) digunakan pada proses pencetakan, yang bersifat subtraktif, dan tidak cocok langsung digunakan dalam sistem pengolahan citra berbasis sensor kamera (Aqilah et al., 2023).

Pemisahan channel warna pada RGB sangat penting dalam deteksi warna. Misalnya, untuk mendeteksi warna merah, hanya channel merah yang akan ditonjolkan dan dua channel lainnya diminimalkan. Dengan pemrosesan ini, segmentasi objek berdasarkan warna menjadi lebih efisien.

2.2 Model Warna HSV

Model HSV (Hue, Saturation, Value) digunakan sebagai alternatif RGB untuk mendeteksi warna karena memisahkan komponen warna (hue) dari intensitas (value). Hue menyatakan warna dasar (0–179 derajat di OpenCV), Saturation menunjukkan kejenuhan warna, dan Value mewakili kecerahan (Sinaga & Marpaung, 2020). Model HSV sangat efektif dalam segmentasi warna karena tidak dipengaruhi pencahayaan secara langsung seperti RGB.

Pada praktik ini, HSV digunakan untuk mendeteksi warna merah, hijau, dan biru berdasarkan rentang nilai hue yang telah ditentukan. Pendekatan ini lebih stabil dibanding menggunakan RGB langsung, terutama pada kondisi pencahayaan yang bervariasi.

2.3 Histogram Citra dan Analisisnya

Histogram adalah representasi grafik dari distribusi intensitas piksel pada citra. Dalam pengolahan citra digital, histogram digunakan untuk menilai kualitas pencahayaan, kontras, dan untuk melakukan transformasi seperti equalization (Xue et al., 2020). Histogram RGB terdiri dari tiga kurva, masing-masing mewakili distribusi warna merah, hijau, dan biru.

Analisis histogram dapat menunjukkan apakah citra terlalu gelap, terlalu terang, atau memiliki rentang warna sempit. Dalam tugas praktikum ini, histogram digunakan untuk setiap hasil deteksi warna dan untuk mengevaluasi distribusi warna dominan.

2.4 Thresholding dan Segmentasi Warna

Thresholding adalah metode segmentasi citra yang memisahkan objek dari latar berdasarkan nilai intensitas piksel. Salah satu metode yang digunakan adalah metode Otsu, yang memilih ambang batas secara otomatis dengan memaksimalkan varians antar-kelas (Zheng et al., 2022).

Dalam konteks segmentasi warna, ambang batas diterapkan dalam domain HSV. Threshold tersebut menentukan rentang hue untuk setiap warna (misalnya merah: 0–10 & 160–180), yang digunakan untuk membuat mask (penyaring warna) dengan fungsi `inRange`.

2.5 Peningkatan Kontras dan CLAHE

CLAHE (Contrast Limited Adaptive Histogram Equalization) adalah metode peningkatan kontras yang bekerja pada blok-blok kecil gambar. Teknik ini mengatasi kelemahan histogram equalization biasa yang cenderung memperbesar noise pada gambar dengan kecerahan rendah. CLAHE membatasi penguatan kontras dengan clip limit (Thepade & Pardhi, 2022).

Metode ini sangat efektif untuk memperbaiki citra backlight karena mampu meningkatkan visibilitas pada area gelap tanpa merusak area terang. Dalam praktik, gambar grayscale hasil konversi citra backlight ditingkatkan kontras dan kecerahannya menggunakan CLAHE agar wajah lebih terlihat.

2.6 Perbaikan Citra Backlight

Citra backlight terjadi ketika objek berada di depan sumber cahaya, sehingga profil objek tampak gelap. Untuk mengatasi ini, teknik fusi transparansi atau region of interest (ROI) dapat digunakan untuk meningkatkan kecerahan lokal di area gelap (Xie et al., 2023). Strategi ini mempertahankan detail di area terang sekaligus memunculkan informasi tersembunyi di area bayangan.

Dalam praktik ini, peningkatan dilakukan dengan kombinasi konversi ke grayscale, peningkatan brightness, dan CLAHE.

BAB III

HASIL

3.1 Soal Pertama

1. DETEKSI WARNA PADA CITRA

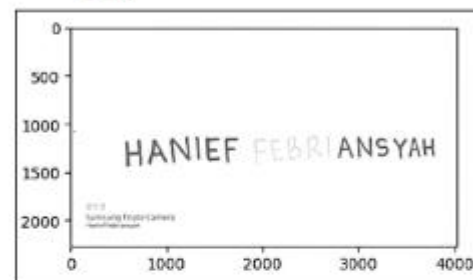
Ketentuan :

- Deteksi warna biru, merah, dan hijau pada gambar lalu tampilkan semua.
- Contoh hasil

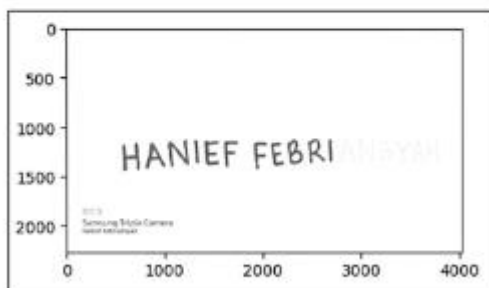
CITRA KONTRAS



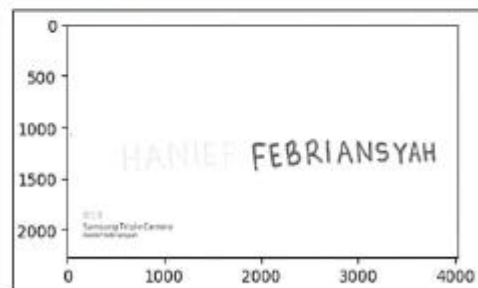
BIRU



MERAH



HIJAU



- Analisislah apa yang terjadi pada citra dan lampirkan hasil analisis pada laporan!
- **Buat histogram** untuk setiap gambar dan analisislah histogram tersebut dan lampirkan pada laporan!

CITRA :



BUKTI LOKASI FOTO :



Tambah Teks

Selasa, 06 Mei 2025 pukul 22.38

[Sesuaikan](#)

IMG_3604

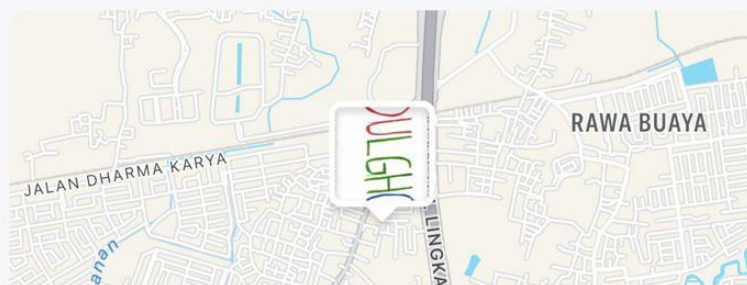
Tangkapan Layar

PNG



Tidak ada informasi lensa

1 MP • 685 × 1581 • 1,4 MB



Kota Administra...t - Duri Kosambi

[Sesuaikan](#)



CODE-NYA :**UTS PRAKTIKUM PCD****SOAL PERTAMA**

```
[27]: import cv2
import matplotlib.pyplot as plt
import numpy as np

[33]: image = cv2.imread("SOAL1.jpg")

[35]: # Memisahkan channel BGR
B, G, R = cv2.split(image)

# Konversi BGR ke RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Membuat subplot untuk menampilkan gambar
fig, axs = plt.subplots(2, 2, figsize=(15, 8))

# Citra RGB
axs[0, 0].imshow(image_rgb)
axs[0, 0].set_title("CITRA KONTRAS")
axs[0, 0].axis('on') # Menampilkan axis ticks
axs[0, 0].grid(False) # Nonaktifkan grid

# Channel Biru
axs[0, 1].imshow(B, cmap='gray')
axs[0, 1].set_title("BIRU")
axs[0, 1].axis('on')
axs[0, 1].grid(False)

# Channel Merah
axs[1, 0].imshow(R, cmap='gray')
axs[1, 0].set_title("MERAH")
axs[1, 0].axis('on')
axs[1, 0].grid(False)

# Channel Hijau
axs[1, 1].imshow(G, cmap='gray')
axs[1, 1].set_title("HIJAU")
axs[1, 1].axis('on')
axs[1, 1].grid(False)

plt.tight_layout()
plt.show()

[37]: # Menampilkan histogram dari RGB dan masing-masing channel
plt.figure(figsize=(20, 5))

plt.subplot(141)
plt.title('Histogram Asli')
plt.hist(image_rgb.ravel(), bins=256, range=[0, 256], color='gray')

plt.subplot(142)
plt.title('Histogram merah')
plt.hist(R.ravel(), bins=256, range=[0, 256], color='r')

plt.subplot(143)
plt.title('Histogram hijau')
plt.hist(G.ravel(), bins=256, range=[0, 256], color='g')

plt.subplot(144)
plt.title('Histogram biru')
plt.hist(B.ravel(), bins=256, range=[0, 256], color='b')

plt.tight_layout()
plt.show()
```

1. Penjelasan Fungsi Setiap Bagian Kode**A. Membaca dan Menyiapkan Citra**

`image = cv2.imread("SOAL1.jpg")`

`B, G, R = cv2.split(image)`

`image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`

- `cv2.imread("SOAL1.jpg")`: Membaca gambar dalam format BGR (standar OpenCV).

- `cv2.split(image)`: Memisahkan gambar menjadi 3 channel: Biru (B), Hijau (G), dan Merah (R).
- `cv2.cvtColor(..., cv2.COLOR_BGR2RGB)`: Mengubah citra ke format RGB untuk ditampilkan dengan matplotlib (karena matplotlib menggunakan RGB, bukan BGR).

B. Menampilkan Gambar dan Tiap Channel

```
fig, axs = plt.subplots(2, 2, figsize=(15, 8))
```

- ☐ Menampilkan 4 subplot: citra asli (RGB) + masing-masing channel (B, G, R).
- ☐ `imshow(..., cmap='gray')`: Menampilkan channel B, G, R sebagai citra grayscale.
- ☐ Ini berguna untuk melihat kontribusi masing-masing warna terhadap keseluruhan citra.

C. Histogram RGB dan Gabungan

```
plt.figure(figsize=(20, 5))
```

```
plt.hist(..., color=...)
```

- Histogram digunakan untuk melihat sebaran intensitas pixel dari 0 (gelap) sampai 255 (terang).
- `ravel()`: Mengubah array 2D menjadi 1D untuk keperluan plotting histogram.
- Setiap subplot menampilkan histogram untuk channel:
 - Merah (R)
 - Hijau (G)
 - Biru (B)
 - Dan satu histogram gabungan ketiganya (`image_rgb.ravel()`).

2. Penjelasan Hasil Dari Analisis Citra:

A. Citra RGB

- Gambar pertama (`image_rgb`) adalah hasil konversi dari BGR ke RGB, karena OpenCV membaca gambar dalam format BGR.
- Gambar ini menampilkan warna aslinya secara natural seperti tampilan manusia melihat (karena RGB adalah standar tampilan layar).

B. Channel Biru (B)

- Menampilkan hanya channel biru, dalam skala abu-abu (grayscale).
- Area yang lebih terang pada gambar ini menunjukkan bagian gambar yang memiliki intensitas biru tinggi, sedangkan area gelap menunjukkan sedikit atau tidak ada warna biru.

C. Channel Merah (R)

- Sama seperti biru, tapi untuk channel merah.
- Area terang adalah area dengan banyak warna merah dalam gambar asli.

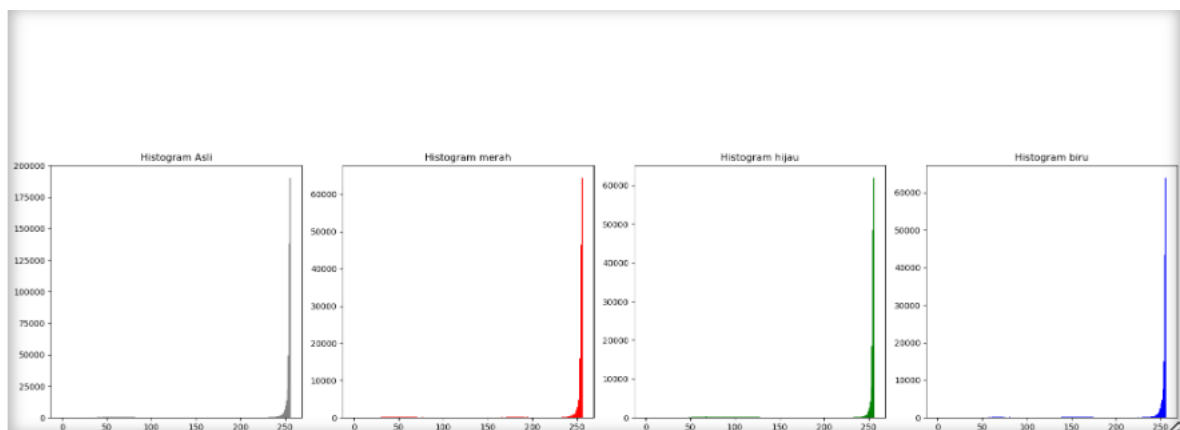
D. Channel Hijau (G)

- Menampilkan channel hijau dalam grayscale.
- Area terang mengindikasikan banyak warna hijau di bagian tersebut.

Kesimpulan bagian citra: Gambar ini dianalisis berdasarkan komposisi warna per channel. Ini membantu kita memahami distribusi warna dalam gambar secara terpisah dan menunjukkan kontribusi masing-masing warna terhadap keseluruhan tampilan.

3. Penjelasan Hasil Dari Analisis Histogram :

Histogram yang dihasilkan setelah run :



A. Histogram Asli (Gabungan RGB)

- Ciri utama: Hampir seluruh nilai piksel terkonsentrasi di area paling kanan (sekitar nilai 250).
- Interpretasi:
 - Gambar ini sangat terang atau mendekati putih secara keseluruhan.
 - Hampir tidak ada piksel dengan nilai gelap (0–200).
 - Ini menandakan gambar memiliki kontras yang rendah karena tidak ada variasi signifikan pada tingkat kecerahan.

B. Histogram Merah

- Ciri utama: Nilai merah sangat tinggi, mayoritas berada di kisaran 245–255.
- Interpretasi:
 - Warna merah sangat mendominasi citra.
 - Area terang gambar sangat dipengaruhi oleh warna merah.
 - Citra kemungkinan memiliki pencahayaan dominan kemerahan, atau pencahayaan putih dengan komponen merah tinggi.

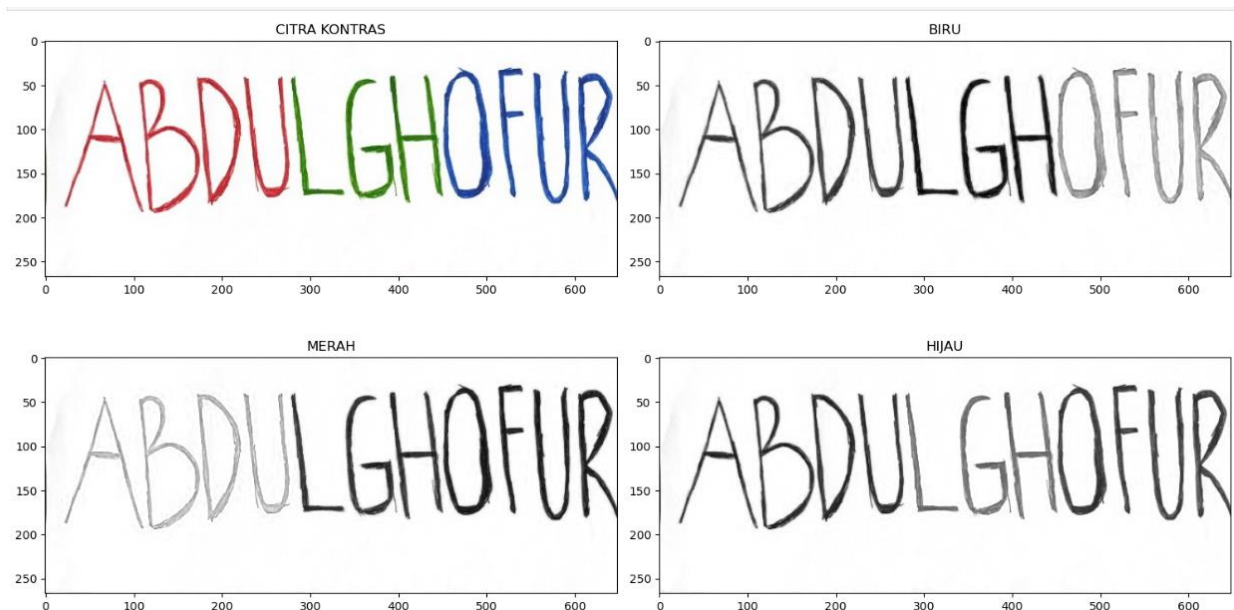
C. Histogram Hijau

- Ciri utama: Hampir sama dengan merah, namun tingkatnya sedikit lebih rendah.
- Interpretasi:
 - Hijau juga cukup dominan, tetapi sedikit di bawah merah.
 - Masih memperkuat bahwa gambar terang dan mendekati putih, karena putih adalah kombinasi dari RGB tinggi.

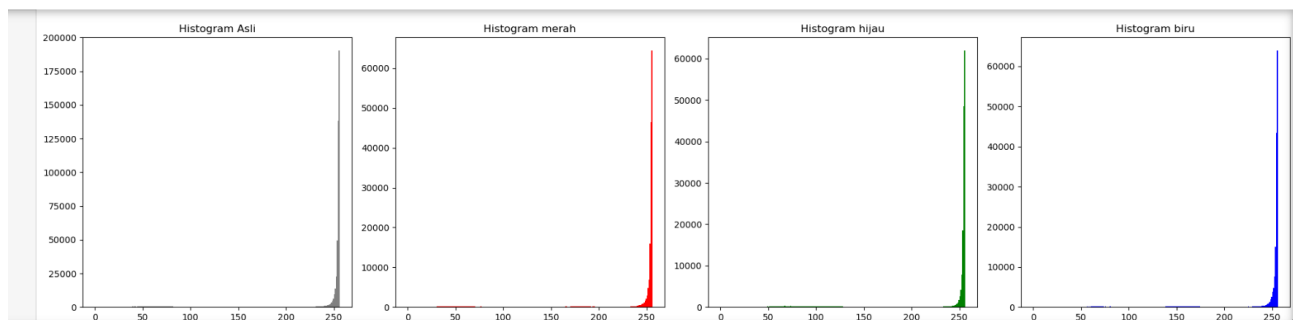
D. Histogram Biru

- Ciri utama: Distribusinya serupa dengan hijau, dengan puncak pada nilai tinggi.
- Interpretasi:
 - Biru juga berkontribusi besar, tetapi intensitasnya sedikit di bawah merah.
 - Ini menandakan bahwa gambar sangat cerah dengan bias sedikit ke arah merah.

HASIL SETELAH RUN:



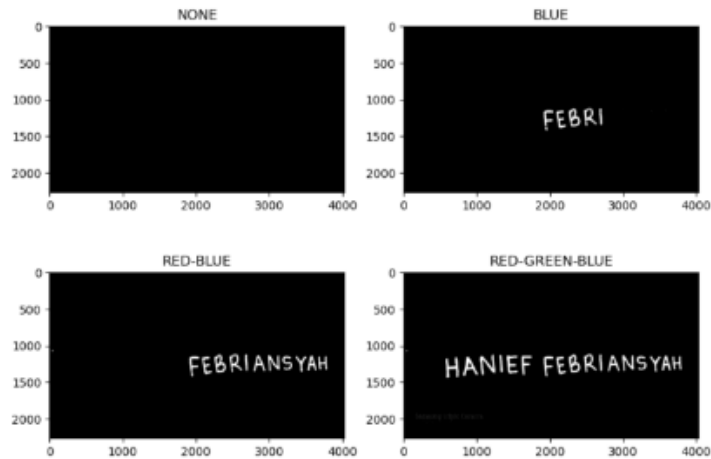
HISTOGRAM :



3.2 SOAL KEDUA

2. CARILAH DAN URUTKAN AMBANG BATAS TERKECIL SAMPAI DENGAN TERBESAR

- o Carilah nilai ambang batas citra untuk dapat menampilkan kategori warna pada citra!
- o Contoh hasil:



- o Lampirkan nilai ambang batas yang didapat dan jelaskan alasan kenapa mendapatkan nilai ambang batas tersebut pada laporan!

CODE-NYA :

SOAL KEDUA

```
[39]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load an image first
color_image = cv2.imread('SOAL1.jpg') # Replace with your image path

# Now proceed with your color processing
hsv = cv2.cvtColor(color_image, cv2.COLOR_BGR2HSV)

lower_blue = np.array([100, 50, 50])
upper_blue = np.array([130, 255, 255])

mask = cv2.inRange(hsv, lower_blue, upper_blue)

hasil = cv2.bitwise_and(color_image, color_image, mask=mask)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(cv2.cvtColor(color_image, cv2.COLOR_BGR2RGB))
axes[0].set_title('Citra Asli')

axes[1].imshow(mask, cmap='gray')
axes[1].set_title('Blue')

plt.show()
```

```

: # Load an image
color_image = cv2.imread('SOAL1.jpg')

# Convert from BGR to RGB for correct display in matplotlib
color_image_rgb = cv2.cvtColor(color_image, cv2.COLOR_BGR2RGB)

# Convert to HSV for color detection
hsv = cv2.cvtColor(color_image, cv2.COLOR_BGR2HSV)

# Define blue range in HSV
lower_blue = np.array([100, 50, 50])
upper_blue = np.array([130, 255, 255])

# Define red ranges in HSV (red wraps around in HSV, so we need two ranges)
lower_red1 = np.array([0, 50, 50])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 50, 50])
upper_red2 = np.array([180, 255, 255])

# Create masks
mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)
mask_red1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask_red2 = cv2.inRange(hsv, lower_red2, upper_red2)

# Combine the two red masks
mask_red = cv2.bitwise_or(mask_red1, mask_red2)

# Create combined red-blue mask
mask_combined = cv2.bitwise_or(mask_blue, mask_red)

# Create plots with just two panels
fig, axs = plt.subplots(1, 2, figsize=(10, 5))

# Show original image (in RGB)
axs[0].imshow(color_image_rgb)
axs[0].set_title('Citra Asli')
axs[0].axis('off') # Hide axes

# Show combined red-blue mask
axs[1].imshow(mask_combined, cmap='gray')
axs[1].set_title('Red-Blue')
axs[1].axis('off') # Hide axes

plt.tight_layout() # Adjust spacing between subplots
plt.show()

```

```

hsv = cv2.cvtColor(color_image, cv2.COLOR_BGR2HSV)

lower_blue = np.array([100, 50, 50])
upper_blue = np.array([130, 255, 255])

lower_red1 = np.array([0, 50, 50])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([170, 50, 50])
upper_red2 = np.array([180, 255, 255])

lower_green = np.array([35, 50, 50])
upper_green = np.array([85, 255, 255])

mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)
mask_red1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask_red2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask_green = cv2.inRange(hsv, lower_green, upper_green)

mask_combined = cv2.bitwise_or(mask_blue, mask_red1)
mask_combined = cv2.bitwise_or(mask_combined, mask_red2)
mask_combined = cv2.bitwise_or(mask_combined, mask_green)

hasil = cv2.bitwise_and(color_image, color_image, mask=mask_combined)

fig, axs = plt.subplots(1, 2, figsize=(10, 5))

axs[0].imshow(cv2.cvtColor(color_image, cv2.COLOR_BGR2RGB))
axs[0].set_title('Citra Asli')

axs[1].imshow(mask_combined, cmap='gray')
axs[1].set_title('Red-Green-Blue')

plt.show()

```

```
def display_image(image, title="Image"):
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title(title)
    plt.axis('on')
    plt.show()

hsv_image = cv2.imread('SOAL1.jpg', cv2.IMREAD_COLOR)
hsv_image = cv2.cvtColor(hsv_image, cv2.COLOR_BGR2HSV)

lower_blue = np.array([100, 50, 50])
upper_blue = np.array([130, 255, 255])

lower_red1 = np.array([0, 50, 50])
upper_red1 = np.array([10, 255, 255])

lower_red2 = np.array([170, 50, 50])
upper_red2 = np.array([180, 255, 255])

lower_green = np.array([50, 50, 50])
upper_green = np.array([80, 255, 255])

blue_mask = cv2.inRange(hsv_image, lower_blue, upper_blue)
red_mask1 = cv2.inRange(hsv_image, lower_red1, upper_red1)
red_mask2 = cv2.inRange(hsv_image, lower_red2, upper_red2)
green_mask = cv2.inRange(hsv_image, lower_green, upper_green)

red_blue_mask = cv2.bitwise_or(red_mask1, red_mask2)
combined_mask = cv2.bitwise_or(blue_mask, cv2.bitwise_or(red_blue_mask, green_mask))

black_image = np.zeros_like(hsv_image, dtype=np.uint8)

black_detected_image = cv2.bitwise_and(black_image, black_image, mask=combined_mask)

display_image(black_detected_image, "None")
```

```
# Tambahkan ini sebelum plotting
mask_red_blue = cv2.bitwise_or(mask_red, mask_blue)

fig, axs = plt.subplots(2, 2, figsize=(12, 8))

axs[0, 0].imshow(cv2.bitwise_and(black_image, black_image, mask=combined_mask))
axs[0, 0].set_title('None')

axs[0, 1].imshow(mask_blue, cmap='gray')
axs[0, 1].set_title('Blue')

axs[1, 0].imshow(mask_red_blue, cmap='gray')
axs[1, 0].set_title('Red-Blue')

axs[1, 1].imshow(mask_combined, cmap='gray')
axs[1, 1].set_title('Red-Green-Blue')

for ax in axs.flat:
    ax.axis('on')

plt.tight_layout()
plt.show()
```

PENJELASAN HASIL AMBANG BATAS:

Warna	Hue (H)	Saturation (S)	Value (V)
Merah	0–10 dan 170–180	50–255	50–255
Hijau	35–85	50–255	50–255
Biru	100–130	50–255	50–255

Nilai-nilai di atas adalah **nilai ambang batas (threshold)** HSV yang digunakan dalam kode

Penjelasan nya :

Merah (Red):

Karena warna merah berada di kedua ujung spektrum Hue (0 dan 180), dibutuhkan **dua rentang**:

```
lower_red1 = np.array([0, 50, 50])
```

```
upper_red1 = np.array([10, 255, 255])
```

```
lower_red2 = np.array([170, 50, 50])
```

```
upper_red2 = np.array([180, 255, 255])
```

Alasan: Merah "melingkar" di ujung skala HSV (0–180), jadi perlu dua rentang untuk mencakup semua rona merah.

Hijau (Green):

```
lower_green = np.array([35, 50, 50])
```

```
upper_green = np.array([85, 255, 255])
```

Alasan: Hue untuk hijau biasanya berkisar antara **60**, jadi rentang 35–85 mencakup semua variasi hijau dari kehijauan muda hingga hijau tua.

Biru (Blue):

```
lower_blue = np.array([100, 50, 50])
```

```
upper_blue = np.array([130, 255, 255])
```

Alasan: Biru memiliki hue sekitar **120**, jadi rentang 100–130 sudah sangat ideal untuk mendeteksi nuansa biru seperti cyan atau navy.

Alasan Saya Menggunakan Saturation dan Value 50–255.

S = 50–255

V = 50–255

Alasan:

Saturation (S) = seberapa kuat warnanya. Di bawah 50 biasanya terlalu abu-abu (kurang jenuh).

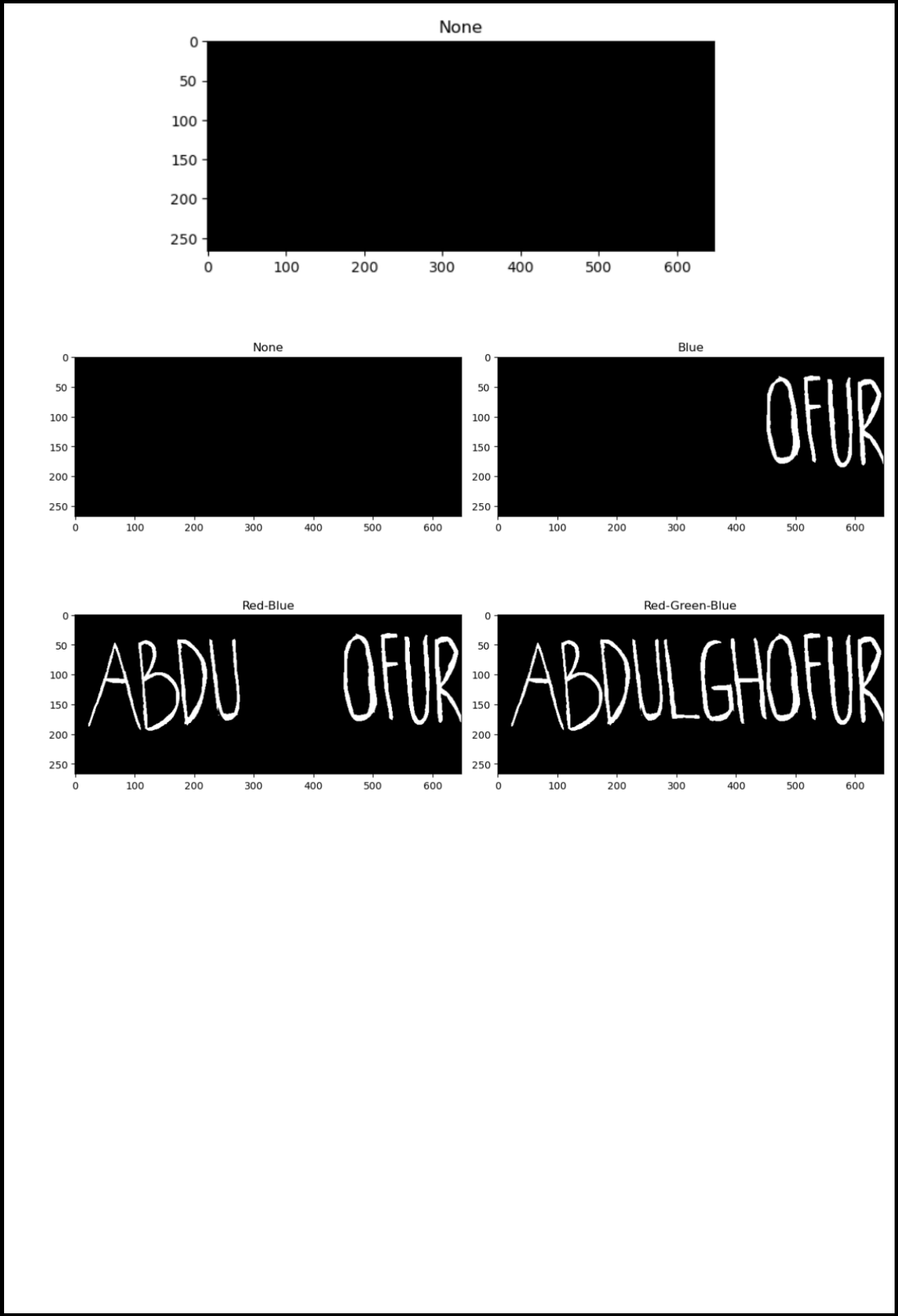
Value (V) = seberapa terang warnanya. Nilai rendah (di bawah 50) sering kali terlalu gelap untuk dideteksi jelas.

Kesimpulan :

Warna	Lower HSV	Upper HSV	Alasan Pemilihan
Merah	[0, 50, 50] dan [170, 50, 50]	[10, 255, 255] dan [180, 255, 255]	Rentang ganda karena merah ada di ujung spektrum HSV
Hijau	[35, 50, 50]	[85, 255, 255]	Rentang hijau alami dari muda ke tua
Biru	[100, 50, 50]	[130, 255, 255]	Cakupan biru muda hingga tua

HASIL SETELAH RUN :





3.3 SOAL KETIGA

SOAL :

3. Memperbaiki Gambar *Backlight*

- Ambillah foto diri Anda dengan posisi menghadap langsung ke kamera dan membelakangi sumber cahaya matahari yang terang (*backlight*).
- Gunakan logika serta teknik-teknik pengolahan citra yang telah Anda pelajari sebelumnya untuk mengolah gambar tersebut. Fokus utama adalah memperbaiki tampilan profil wajah atau tubuh Anda yang cenderung gelap akibat efek *backlight*.
- Lakukan konversi gambar menjadi *grayscale*, kemudian tingkatkan kecerahan dan kontras khususnya pada area profil Anda sehingga lebih menonjol dibandingkan latar belakang yang terang.
- Penilaian akan difokuskan pada seberapa efektif Anda membuat area profil menjadi pusat perhatian (fokus utama) dibandingkan dengan latar belakangnya.
- Pengurangan nilai akan diberikan pada gambar yang mengalami efek *color burn* (terbakar warna) secara berlebihan. Namun, toleransi masih diberikan untuk efek *color burn* yang wajar dan tidak mengganggu kualitas visual citra.

CITRA :



BUKTI LOKASI FOTO :



Tambah Teks

Minggu, 04 Mei 2025 pukul 16.14

Sesuaikan

IMG_3578

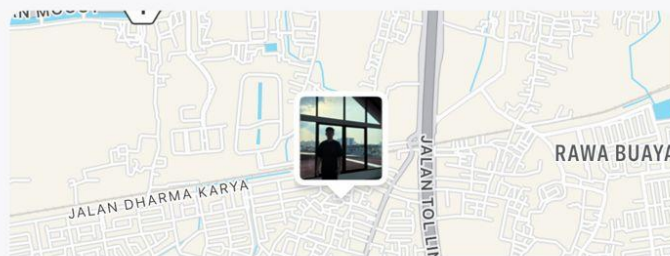
Apple iPhone 13

HEIF

Kamera Lebar — 26 mm $f1.6$

9 MP • 4032 × 2268 • 1,9 MB

ISO 50 | 26 mm | -0,3 ev | $f1.6$ | 1/1140 s



Kota Administra...t - Duri Kosambi >

Sesuaikan



CODE-NYA :

```

import cv2
import matplotlib.pyplot as plt
import numpy as np

# Baca gambar asli dari file
path = "SOAL2.jpg"
img = cv2.imread(path)

# Konversi BGR ke RGB untuk ditampilkan dengan matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Konversi ke grayscale
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Pencerahan gambar gray (ditambah nilai brightness)
bright_gray = cv2.convertScaleAbs(img_gray, alpha=1, beta=50) # beta menambah kecerahan

# Penambahan kontras pada gambar gray
contrast_gray = cv2.convertScaleAbs(img_gray, alpha=1.5, beta=0) # alpha meningkatkan kontras

# Kombinasi: Dipercah + Diperkontras
bright_contrast_gray = cv2.convertScaleAbs(img_gray, alpha=1.5, beta=50)

plt.figure(figsize=(20, 30)) # Lebar diperbesar

plt.subplot(6, 1, 1)
plt.imshow(img_rgb)
plt.title("Gambar Asli", fontsize=18)
plt.axis("off")

plt.subplot(6, 1, 2)
plt.imshow(img_gray, cmap='gray')
plt.title("Gambar Gray", fontsize=18)
plt.axis("off")

plt.subplot(6, 1, 3)
plt.imshow(bright_gray, cmap='gray')
plt.title("Gambar Gray yang Dipercah", fontsize=18)
plt.axis("off")

plt.subplot(6, 1, 4)
plt.imshow(contrast_gray, cmap='gray')
plt.title("Gambar Gray yang Diperkontras", fontsize=18)
plt.axis("off")

plt.subplot(6, 1, 5)
plt.imshow(bright_contrast_gray, cmap='gray')
plt.title("Gambar Gray Dipercah & Diperkontras", fontsize=18)
plt.axis("off")

plt.subplots_adjust(hspace=0.4) # Atur jarak antar gambar
plt.show()

```

PENJELASAN DAN ANALISIS :**1. Import Library**

import cv2

import matplotlib.pyplot as plt

import numpy as np

Fungsi:

cv2 (OpenCV): Digunakan untuk memproses gambar (membaca, mengubah warna, meningkatkan kualitas, dll).

matplotlib.pyplot: Untuk menampilkan gambar dalam bentuk plot.

numpy: Umumnya digunakan untuk manipulasi array, tapi di kode ini tidak digunakan secara langsung.

2. Membaca dan Mengonversi Gambar

```
path = "SOAL2.jpg"
```

```
img = cv2.imread(path)
```

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Fungsi:

cv2.imread(path): Membaca gambar dari file.

cv2.cvtColor(img, cv2.COLOR_BGR2RGB): OpenCV membaca gambar dalam format BGR, sedangkan matplotlib butuh format RGB untuk menampilkan warna dengan benar.

3. Konversi ke Grayscale

```
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Fungsi: Mengubah gambar warna menjadi grayscale. Ini menghilangkan informasi warna dan hanya mempertahankan intensitas cahaya (kecerahan).

4. Pencerahan Gambar Gray

```
bright_gray = cv2.convertScaleAbs(img_gray, alpha=1, beta=50)
```

Fungsi:

alpha = 1: Tidak mengubah kontras.

beta = 50: Menambah nilai intensitas pixel sebesar 50. Ini membuat gambar lebih **terang**.

Analisis: Area gelap menjadi lebih cerah, detail yang sebelumnya sulit dilihat di bagian gelap akan muncul lebih jelas.

5. Penambahan Kontras Gambar Gray

```
contrast_gray = cv2.convertScaleAbs(img_gray, alpha=1.5, beta=0)
```

Fungsi:

alpha = 1.5: Meningkatkan kontras dengan mengalikan nilai intensitas piksel.

beta = 0: Tidak ada perubahan pencahayaan.

Analisis: Area terang jadi lebih terang, area gelap menjadi lebih gelap.

Perbedaan antar area makin jelas, meningkatkan ketajaman visual, tapi bisa menyebabkan kehilangan detail di area sangat terang/gelap.

6. Gabungan Pencerahan dan Kontras

```
bright_contrast_gray = cv2.convertScaleAbs(img_gray, alpha=1.5, beta=50)
```

Fungsi:

alpha = 1.5: Meningkatkan kontras.

beta = 50: Menambah kecerahan.

Analisis: Gambar jadi **lebih cerah dan lebih tajam**.

untuk memperjelas detail di kondisi cahaya redup atau backlight, dapat juga menyebabkan highlight yang terlalu terang.

7. Menampilkan Gambar

```
plt.figure(figsize=(20, 30))
```

```
# ... subplot kode ...
```

```
plt.show()
```

Fungsi: Menampilkan semua hasil gambar transformasi (asli, grayscale, cerah, kontras, dan kombinasi) secara vertikal.

figsize=(20, 30): Ukuran tampilan gambar.

subplot(6,1,n): Menyusun gambar dalam 6 baris.

HASIL SETELAH RUN:

Gambar Asli



Gambar Gray



Gambar Gray yang Dipercerah



Gambar Gray yang Diperkontras



Gambar Gray Dipercerah & Diperkontras



BAB IV

PENUTUP

Berdasarkan hasil praktikum dan landasan teori yang telah dijabarkan, dapat disimpulkan bahwa pemrosesan citra digital melalui pendekatan berbasis warna dan peningkatan kualitas visual sangat bergantung pada pemahaman terhadap model warna dan distribusi intensitas piksel.

Penggunaan model HSV terbukti lebih stabil dan akurat dibandingkan model RGB untuk segmentasi warna karena HSV memisahkan informasi warna (hue) dari kecerahan (value), sehingga mendeteksi warna seperti merah, hijau, dan biru menjadi lebih presisi meskipun terdapat variasi pencahayaan.

Pemisahan channel warna RGB maupun konversi ke HSV menjadi teknik dasar penting dalam pengolahan citra awal, khususnya untuk keperluan klasifikasi, segmentasi, dan analisis visual objek berbasis warna. Histogram RGB memberikan gambaran distribusi intensitas tiap channel warna, yang berguna dalam mengevaluasi dominansi warna dan kualitas pencahayaan pada citra.

Selanjutnya, penerapan metode thresholding (terutama Otsu) pada citra HSV memberikan hasil segmentasi warna yang optimal. Pendekatan ini mampu menentukan ambang batas secara adaptif untuk membedakan warna-warna target dengan latar, sehingga hasil deteksi menjadi lebih selektif dan bebas noise.

Pada bagian peningkatan kualitas citra, khususnya dalam kondisi backlight, metode CLAHE memberikan solusi yang efektif dengan meningkatkan kontras lokal pada area gelap tanpa menyebabkan overexposure pada latar terang. Dengan demikian, objek utama (seperti wajah dalam foto backlight) dapat ditonjolkan secara visual tanpa mengorbankan detail keseluruhan citra.

Keseluruhan praktikum ini menunjukkan pentingnya integrasi antara teori dasar ruang warna, analisis histogram, thresholding adaptif, serta teknik peningkatan kontras dalam menyelesaikan berbagai persoalan pada pengolahan citra digital. Pengetahuan ini menjadi landasan yang kuat untuk pengembangan aplikasi lebih lanjut di bidang computer vision, seperti object detection, image classification, dan autonomous vision systems.

.

DAFTAR PUSTAKA

1. Sinaga, P., & Marpaung, Y. (2020). Implementation of HSV-based Thresholding Method for Iris Detection. *Journal of Computer and Network Applications*, 2(1). Retrieved from <https://www.jurnal.itscience.org/index.php/CNAPC/article/download/939/632/2595>
2. Chromaticity and image processing applied to determine the erosion. (2025). *Heliyon*, 11(5), e25095. <https://doi.org/10.1016/j.heliyon.2025.e25095>
3. Zheng, J., Gao, Y., Zhang, H., Lei, Y., & Zhang, J. (2022). OTSU Multi-Threshold Image Segmentation Based on Improved Particle Swarm Algorithm. *Applied Sciences*, 12(22), 11514. <https://doi.org/10.3390/app122211514>
4. Improving the segmentation of digital images by using a modified Otsu function. (2023). *Multimedia Tools and Applications*. <https://doi.org/10.1007/s11042-023-15129-y>
5. Comparison of Image Enhancement Techniques: HE, LHE, CLAHE. (2020). *ISPRS Archives, XLIV-M-2*, 45–50. <https://doi.org/10.5194/isprs-archives-XLIV-M-2-2020-45-2020>
6. Xue, Z., et al. (2020). Dynamic histogram equalization for contrast enhancement for digital images. *Applied Soft Computing*, 89, 106114. <https://doi.org/10.1016/j.asoc.2020.106114>
7. Enhancement of satellite images based on CLAHE and augmented Elk Herd Optimizer. (2024). *Soft Computing*. <https://doi.org/10.1007/s10462-024-11022-8>
8. Thepade, S., & Pardhi, P. M. (2022). Contrast enhancement with brightness preservation of low light images using blending of CLAHE and BPDHE. *International Journal of Information Technology*, 14(3), 1345–1351. <https://doi.org/10.1007/s41870-022-00951-4>
9. An improved fusion strategy based on transparency-guided backlit image enhancement. (2024). *ResearchGate*. Retrieved from <https://www.researchgate.net/publication/384738447>
10. Contrast enhancement of region of interest of backlit image for low intensity regions. (2023). *Computers in Biology and Medicine*, 165, 107374. <https://doi.org/10.1016/j.compbiomed.2023.107374>